Testes Automatizados — SIAS BRASIL

🧪 Sobre o projeto

Este repositório contém o **Projeto de Automação do SIAS DA SAS BRASIL** que foi desenvolvido com **Robot Framework** e **Selenium Library** para auxiliar o time de desenvolvimento e qualidade durante jornadas de testes para validação de diversos cenários críticos e funcionamento correto dos principais módulos do SIAS. Com objetivo de garantir **confiabilidade**, **repetibilidade** e **facilidade de execução**.

📋 Pré-requisitos

Antes de começar, é necessário instalar:

1. **Python 3.11+**
   - Baixe o executável do `Python` através do link acima, e no Windows durante a instalação deixe marcada a opção *"Add Python to PATH"*.

> ⓘ **Atenção!**
>
> Para baixar, role a página até o final e procure pela versão de instalação compatível com seu sistema operacional. Exemplo: "Windows installer (64-bit)"

2. **Google Chrome for Testing**
   - Baixe a versão `Stable` do `Chrome for tests` compatível com seu sistema operacional.
   - Extraia os arquivos e coloque na raiz da pasta `\drivers`:

```
Projeto/
└── drivers/                    <- Aqui
    └── chrome-win64            # Pasta chrome correta (versão 64x)
```

3. **ChromeDriver**
   - Baixe também a versão `Stable` mais recente do `Chromedriver` compatível com seu sistema operacional e com o *Chrome for tests* que você baixou anteriormente.
   - Extraia os arquivos e coloque na raiz da pasta `\drivers`:

```
Projeto/
└── drivers/                    <- Aqui
```

```
    ├── chrome-win64          # Pasta chrome correta (versão 64x)
    └── chromedriver-win64    # Pasta ChromeDriver correta (versão 64x)
```

> ⚡ **Atenção!**
>
> A configuração destes pré-requisitos é crucial para os próximos passos e o projeto não funcionará sem os drivers do `Chrome` nos locais corretos e sem o `python` devidamente instalado.

📦 Dependências

O projeto utiliza as seguintes dependências, que também já estão no arquivo de configuração automática `requirements.txt`:

```
robotframework==7.0
robotframework-seleniumlibrary==6.3.0
selenium==4.21.0
webdriver-manager==4.0.2
```

Siga os passos de configuração para instalar corretamente as dependências na criação do ambiente virtual *(passo 4 em Configuração e Execução)* .

📂 Estrutura do Projeto

```
projeto-sias-partners/
│
├── setup.bat                   # Script opcional de configuração no Windows
│
├── drivers/                    # Binários do navegador e driver
│   ├── chrome-win64            # Pasta chrome correta (versão 64x)
│   │   └── chrome.exe          # Executável Chrome for Testing (manual)
│   │
│   ├── chromedriver-win64      # Pasta ChromeDriver correta (versão 64x)
│   │   └── chromedriver.exe    # ExecutávelChromeDriver (manual)
│   ├── anexos                  # Assets usados por alguns testes
│   └── requirements.txt        # Arquivo de configuração dependências
│
├── logs/                       # Relatórios e logs de execução
│
├── resources/                  # Keywords, variáveis e configurações
│   ├── *.resource
```

```
│   └── scripts/                # Scripts Python auxiliares
│       └── *.py
│
└── tests/                      # Casos de teste (.robot)
    ├── *.robot
    └── __init__.robot          # Configuração de suíte principal
```

🚀 Configuração e Execução

1️⃣ Clonar o repositório

```
git clone https://seu-repositorio.git
cd setup
```

2️⃣ Criar e ativar ambiente virtual (venv)

Windows (PowerShell)

```
python -m venv venv
venv\Scripts\Activate.ps1
```

Windows (CMD)

```
python -m venv venv
venv\Scripts\activate
```

Linux/Mac

```
python3 -m venv venv
source venv/bin/activate
```

3️⃣ Instalar dependências

```
pip install -r drivers\requirements.txt
```

4️⃣ Colocar drivers do Chrome na pasta correta

- Baixe **Chrome for Testing** e **ChromeDriver** da **mesma versão** Stable.
- Salve-os descompactados e coloque na pasta `drivers/` na raiz do projeto.

> ⚠️ **Aviso**

> Confira o nome correto da pasta de drivers e certifique-se que estão com os caminhos como indicado na ***estrutura do projeto***, após baixar e extrair.

Se já houver realizado os pré-requisitos, basta ignorar este passo.

5️⃣ Executando testes - Padrão

Todos os testes

```
robot tests/
```

Teste específico

```
robot tests/login.robot
```

6️⃣ Passos de Execução opcional

O projeto pode ser utilizado baixando e instalando uma IDE como o **Visual Studio Code**. Para esse uso, você precisará instalar a extensão do Robot que pode ser instalado a partir da aba de **extensões** dentro do VS Code ou na página oficial do **RobotFramework**.

✅ Execução Rápida

O projeto dispõe de uma configuração automática através do `setup.bat` disponível na raiz do projeto. Basta executar o arquivo que ele irá configurar automaticamente o `.venv/`, baixar e instalar as dependências do `requirements.txt` e iniciará um **Console Interativo** onde você pode rodar os testes de maneira prática.

> ⚠️ **Aviso**
>
> Para que as configurações e o **Console Interativo** funcionem através do `setup.bat` é necessário seguir corretamente os passos de pré-requisito.

📊 Relatórios

Após a execução, serão gerados:

- `log.html` → Detalhes da execução
- `report.html` → Resumo dos testes

- `output.xml` → Saída padrão do Robot

Todos ficam dentro da pasta `logs/`.

> **Importante**
>
> Utilizando o `setup.bat` o `log.html` irá abrir automaticamente no seu navegador, após o final de uma execução sendo bem sucedida ou com falha.

> **Aviso**
>
> A pasta `logs/` é limpa automaticamente após o início de um novo test. Certifique-se de fazer backups caso necessário.

📌 Boas práticas

- Sempre rodar com o **venv ativo** para evitar conflitos.
- Manter **versões compatíveis** do Chrome e ChromeDriver.
- Usar **tags** para filtrar e organizar testes:

```
robot --include login tests/
```

🦾 Suporte

Encontrou problemas? Solicite manutenção dos arquivos ao nosso time responsável:

Suporte de Desenvolvimento Jr. 📧 evaldo.junior@sasbrasil.org.br	**Estagiário TI:** 📧 joaovitor.araujo@sasbrasil.org.br

---

🗂️ Documentações

Robot Framework
Keywords Robot e Selenium